



Digital Nurture 5.0

What is the Singleton Design Pattern?

The **Singleton Design Pattern** is a **Creational Design Pattern** that ensures a class has **only one instance** throughout the application's lifecycle and provides a **global point of access** to that instance.

It is useful when exactly one object is required to coordinate actions across the system.

Characteristics

- Only one object of the class is created.
- The constructor is declared **private**.
- A **static instance** of the class is maintained.
- A **public static method** returns the single instance.

Advantages

- Saves memory by preventing multiple object creation.
- Provides controlled access to a shared resource.
- Easy to implement and widely used.
- Suitable for logging, configuration settings, cache, and database connections.

Applications

- Database Connection
- Logger Class

Algorithm

1. Create a class named **Singleton**.
2. Make its constructor **private**.
3. Create a private static object of the class.
4. Create a public static method **getInstance()** to return the same object.
5. In the main method, call **getInstance()** multiple times.
6. Verify that both references point to the same object.

CODE :

```
Exercise 1: Implementing the Singleton Pattern > Singleton_Pattern.java > Singleton > getInstance()
1  class Singleton {
2
3
4      private static Singleton instance;
5
6
7      private Singleton() {
8          System.out.println(x: "Singleton Object Created");
9      }
10
11
12  public static Singleton getInstance() {
13      if (instance == null) {
14          instance = new Singleton();
15      }
16      return instance;
17  }
18
19  public void display() {
20      System.out.println(x: "Welcome to Singleton Design Pattern");
21  }
22  }
23
24  public class Singleton_Pattern {
25
26      public static void main(String[] args) {
27
28          Singleton obj1 = Singleton.getInstance();
29          Singleton obj2 = Singleton.getInstance();
30
31          obj1.display();
32
33          System.out.println("Object 1 hashCode : " + obj1.hashCode());
34          System.out.println("Object 2 hashCode : " + obj2.hashCode());
35
36          if (obj1 == obj2) {
37              System.out.println(x: "Both references point to the same object.");
38          } else {
39              System.out.println(x: "Different objects are created.");
40          }
41      }
42  }
```

OUTPUT:

```
Singleton_Pattern
Singleton Object Created
Welcome to Singleton Design Pattern
Object 1 hashCode : 1829164700
Object 2 hashCode : 1829164700
Both references point to the same object.
```